1. Moveable Shapes Simulation • Define an interface Movable with methods: moveUp(), moveDown(), moveLeft(), moveRight(). • Implement classes: o MovablePoint(x, y, xSpeed, ySpeed) implements Movable o MovableCircle(radius, center: MovablePoint) o MovableRectangle(topLeft: MovablePoint, bottomRight: MovablePoint) (ensuring both points have same speed) • Provide toString() to display positions. • In main(), create a few objects and call move methods to simulate motion.

Code:

```
package day_03;

interface Movable {
   void moveUp();
   void moveDown();
   void moveLeft();
   void moveRight();
}

class MovablePoint implements Movable {
   int x, y, xSpeed, ySpeed;
   MovablePoint(int x, int y, int xSpeed, int ySpeed) {
      this.x = x;
      this.y = y;
      this.xSpeed = xSpeed;
      this.ySpeed = ySpeed;
   }
   public void moveUp() {
      y += ySpeed;
   }
   public void moveDown() {
      y -= ySpeed;
   }
   public void moveLeft() {
      x -= xSpeed;
   }
   public void moveRight() {
      x += xSpeed;
```

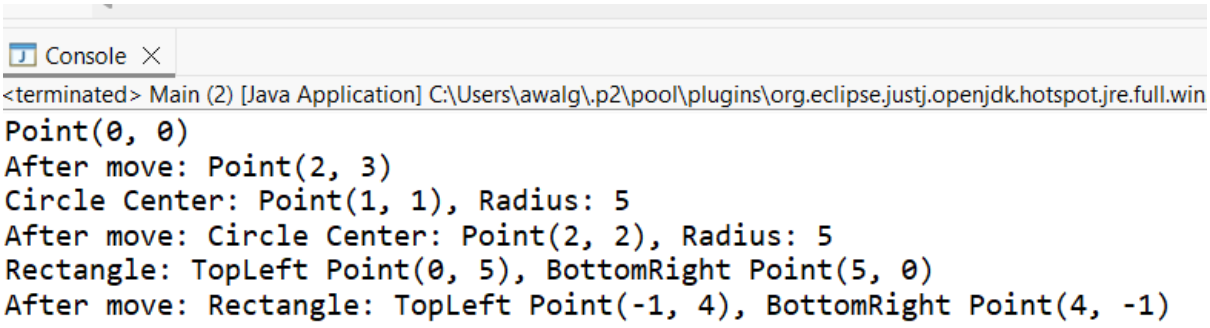
```
    }
    public String toString() {
        return "Point(" + x + ", " + y + ")";
    }
}
class MovableCircle implements Movable {
    int radius;
    MovablePoint center;
    MovableCircle(int radius, MovablePoint center) {
        this.radius = radius;
        this.center = center;
    }
    public void moveUp() { center.moveUp(); }
    public void moveDown() { center.moveDown(); }
    public void moveLeft() { center.moveLeft(); }
    public void moveRight() { center.moveRight(); }

    public String toString() {
        return "Circle Center: " + center + ", Radius: " + radius;
    }
}
class MovableRectangle implements Movable {
    MovablePoint topLeft;
    MovablePoint bottomRight;
    MovableRectangle(MovablePoint topLeft, MovablePoint bottomRight) {
        if (topLeft.xSpeed != bottomRight.xSpeed ||
            topLeft.ySpeed != bottomRight.ySpeed) {
            System.out.println("Speed mismatch!");
        }
        this.topLeft = topLeft;
        this.bottomRight = bottomRight;
```

```
    }
    public void moveUp() {
        topLeft.moveUp();
        bottomRight.moveUp();
    }
    public void moveDown() {
        topLeft.moveDown();
        bottomRight.moveDown();
    }
    public void moveLeft() {
        topLeft.moveLeft();
        bottomRight.moveLeft();
    }
    public void moveRight() {
        topLeft.moveRight();
        bottomRight.moveRight();
    }
    public String toString() {
        return "Rectangle: TopLeft " + topLeft + ", BottomRight " + bottomRight;
    }
}
class Main {
    public static void main(String[] args) {
        MovablePoint p = new MovablePoint(0, 0, 2, 3);
        System.out.println(p);
        p.moveUp();
        p.moveRight();
        System.out.println("After move: " + p);
        MovableCircle c = new MovableCircle(5, new MovablePoint(1, 1, 1, 1));
        System.out.println(c);
        c.moveRight();
```

```
        c.moveUp();
        System.out.println("After move: " + c);
        MovableRectangle r = new MovableRectangle(
                new MovablePoint(0, 5, 1, 1),
                new MovablePoint(5, 0, 1, 1)
        );
        System.out.println(r);
        r.moveDown();
        r.moveLeft();
        System.out.println("After move: " + r);
    }
}
```

Output:



2. 2. Default and Static Methods in Interfaces • Declare interface Polygon with: o double getArea() o default method default double getPerimeter(int... sides) that computes sum of sides o a static helper static String shapeInfo() returning a description string • Implement classes Rectangle and Triangle, providing appropriate getArea(). • In Main, call getPerimeter(...) and Polygon.shapeInfo().

Code:

```
package day_03;

interface Polygon {
    double getArea();
    default double getPerimeter(int... sides) {
        int sum = 0;
        for (int side : sides) {
```

```
            sum += side;
        }
        return sum;
    }
    static String shapeInfo() {
        return "Polygons are closed shapes with multiple sides.";
    }
}
class Rectangle implements Polygon {
    int length, breadth;
    Rectangle(int l, int b) {
        length = l;
        breadth = b;
    }
    public double getArea() {
        return length * breadth;
    }
}
class Triangle implements Polygon {
    int base, height;
    Triangle(int b, int h) {
        base = b;
        height = h;
    }
    public double getArea() {
        return 0.5 * base * height;
    }
}
class Main2 {
    public static void main(String[] args) {
        Rectangle r = new Rectangle(10, 5);
```

```
        Triangle t = new Triangle(6, 4);
        System.out.println("Rectangle Area: " + r.getArea());
        System.out.println("Triangle Area: " + t.getArea());
        System.out.println("Rectangle Perimeter: " + r.getPerimeter(10, 5, 10, 5));
        System.out.println("Triangle Perimeter: " + t.getPerimeter(3, 4, 5));
        System.out.println(Polygon.shapeInfo());
    }
}
```

Output: